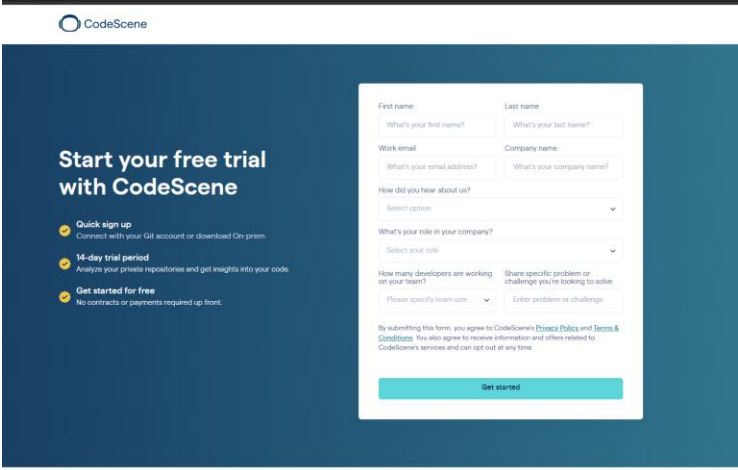


Homework 6 (CodeScene)

1. Creating an account:

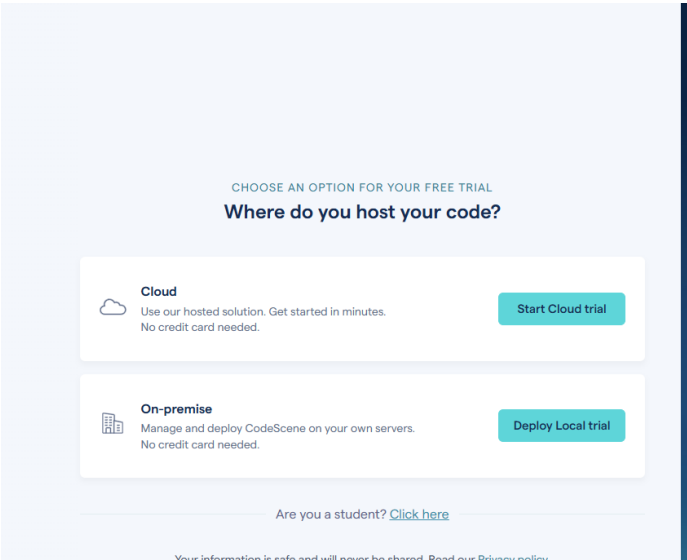


The image shows the CodeScene registration page. On the left, there's a dark blue section with the text "Start your free trial with CodeScene". Below this, there are three bullet points: "Quick sign up" (Connect with your Git account or download On-prem), "14-day trial period" (Analyze your private repositories and get insights into your code), and "Get started for free" (No contracts or payments required up front). On the right, there's a white registration form with fields for "First name", "Last name", "Work email", and "Company name". Below these are dropdown menus for "How did you hear about us?" and "What's your role in your company?". There are also input fields for "How many developers are working on your team?" and "Share specific problem or challenge you're looking to solve". At the bottom of the form is a "Get started" button.

- a.
- b. First step is to start a free trial by signing up on the CodeScene website:
[Start your free trial | CodeScene](#)

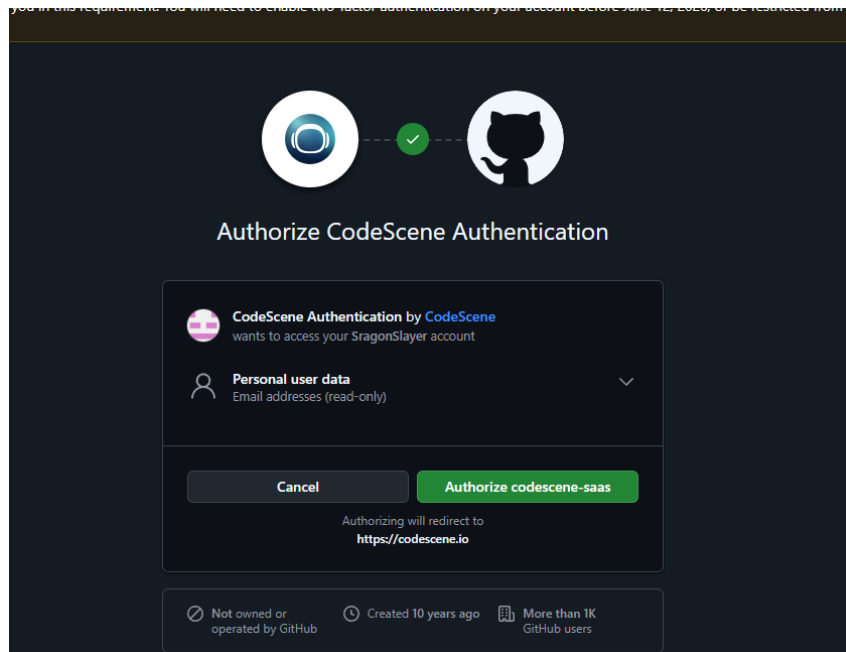
Configuring the AI

2. Next you need to select a cloud base as we hold our code in github:



The image shows the CodeScene trial selection screen. At the top, it says "CHOOSE AN OPTION FOR YOUR FREE TRIAL" and "Where do you host your code?". There are two main options: "Cloud" and "On-premise". The "Cloud" option includes a cloud icon, the text "Use our hosted solution. Get started in minutes. No credit card needed.", and a "Start Cloud trial" button. The "On-premise" option includes a server icon, the text "Manage and deploy CodeScene on your own servers. No credit card needed.", and a "Deploy Local trial" button. At the bottom, there's a link "Are you a student? Click here" and a small disclaimer "Your information is safe and will never be shared. Read our Privacy policy".

3. You will want to grant them access to your github account:



4. Then select which repository you want analyzed:

Select Repositories

Select the repositories you want to analyze in your project.

Want to analyze private repositories? Upgrade your payment plan.

i Your current plan has a limit of max 100 repositories.

Filter visible repositories by regular expression

Example: ^myuser

- ☐ Select all
- ☐ SragonSlayer/academic-codebase
- ☒ SragonSlayer/ClassTesting
- ☐ The-Lucas-Group/Exploration5_4 

5. And also configure the tool to integrate with Pull Requests:

Pull Request Integration
Integrating CodeScene with GitHub Pull Requests requires installing a separate GitHub application.

☐ Enable pull request integration for this project



Code quality gates profile
Select a predefined profile tailored to your team's needs.

Configuration as code
Define custom quality gates for specific code areas or team workflows.

Select quality gates profile
Use global configuration - The Bare Minimum (Minimal Safety Net)

This project is currently using the default global quality gates configuration.

Activate essential protections by enabling all gates in the "Minimal Safety Net." In the PR, these gates display foundational quality checks; learn more in our [documentation](#) about how they operate.

Minimal Safety Net ☒ Prevent Hotspot Decline ☒ New Code is Healthy ☒ Enforce Critical Code Health Rules	Quality Guardians ☐ Enforce Advisory Code Health Rules ☐ Tag Reviewer for Critical Code	Elevate Code Health ☐ Enforce Refactoring Goals ☐ Enforce Suppression Goals
---	--	--

Exclude source branches
A regular expression that specifies which source branches to exclude from the analysis. For example, `release` excludes all branches that have the word `release` in their name.

☐ Only analyze PRs targeting the branch selected in the project repository configuration

Change Coupling Threshold (%)
80
Specifies minimal temporal coupling for the "Absence of Expected Change Pattern" warning.

Results will be presented in Check Runs. CodeScene can also add annotations to your files and/or Pull Request Review.

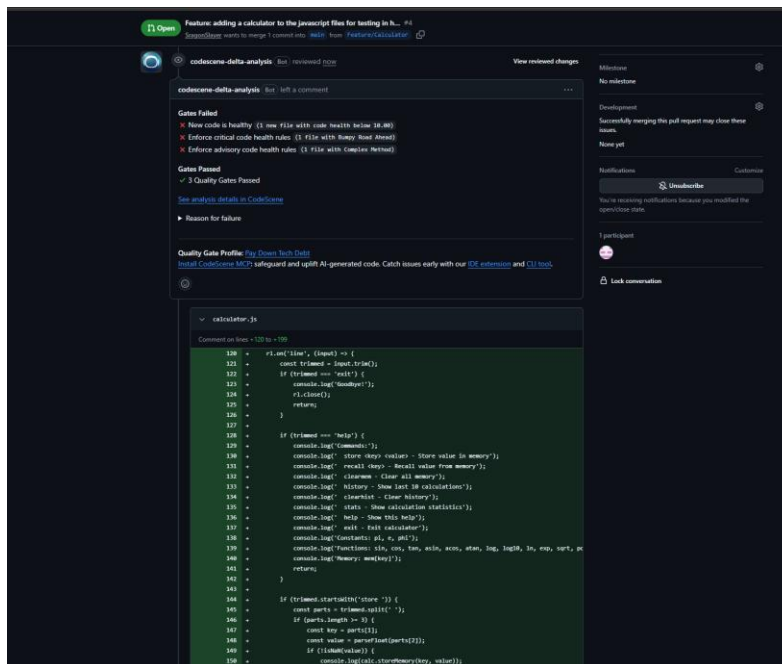
☒ Always comment on all Pull Requests
CodeScene can integrate with your Pull Requests and provide an automated code health check and recommend a review level based on the predicted risk. By default, CodeScene only comments on Pull Requests that a) fail a quality gate, or b) have an increased risk, where we recommend a more detailed review, or c) triggered any warnings. Enabling this checkbox makes CodeScene comment on all Pull Requests, informing on lower risk changes too.

Post individual findings as:

- ☐ Add to Check Run details
- ☐ Post annotations in the Check Run
- ☒ Create a review in the Pull Request

[Update](#)

6. Once the tool is configured, it will automatically review code based on the criteria defined:



```
calculator.js
Comment on line 102 to 109

100 + // eslint('line', (input) => {
101 +   const trimmed = input.trim();
102 +   if (trimmed === 'exit') {
103 +     console.log('Goodbye!');
104 +     // eslint('line', (input) => {
105 +       console.log('');
106 +     }
107 +   }
108 +   if (trimmed === 'help') {
109 +     console.log('Commands:');
110 +     console.log('  store <value> - Store value in memory');
111 +     console.log('  recall <key> - Recall value from memory');
112 +     console.log('  clearmem - Clear all memory');
113 +     console.log('  history - Show last 10 calculations');
114 +     console.log('  clearhist - Clear history');
115 +     console.log('  stats - Show calculation statistics');
116 +     console.log('  help - Show this help');
117 +     console.log('  exit - Exit calculator');
118 +     console.log('Constants: pi, e, phi');
119 +     console.log('Functions: sin, cos, tan, sinh, cosh, atan, log, ln, exp, sqrt, n');
120 +     console.log('Memory: mem[key]');
121 +     return;
122 +   }
123 +   if (trimmed.startsWith('store ')) {
124 +     const parts = trimmed.split(' ');
125 +     if (parts.length > 2) {
126 +       const key = parts[1];
127 +       const value = parts[2].trim();
128 +       if (!isNaN(value)) {
129 +         console.log('storeMemory(key, value)');
130 +       }
131 +     }
132 +   }
133 + }
```

Analysis:

This tool is helpful, but it lacks a lot of qualities needed when updating code. This would be good for low-level linting, catching typos, and some other static code analysis. But it lacks contextual understanding that another developer would normally catch. For example in this PR I have made, it only caught issues with the cyclomatic complexity being above 10,

and nothing else. I am sure this could be expanded with further configuration settings, but I don't think it could replace a humans review.